

Lab 3: Decision Support System for Cardiovascular disease

```
In [1]: from google.colab import drive
import pandas as pd
import seaborn as sns
import numpy as np
import matplotlib.pyplot as plt
```

```
In [2]: drive.mount('/content/drive')
```

Mounted at /content/drive

```
In [3]: # Use path
dataset = pd.read_csv('/content/drive/MyDrive/heart_disease.csv')
```

```
In [4]: dataset.head(5)
```

```
Out[4]:
```

	HeartDisease	BMI	Smoking	AlcoholDrinking	Stroke	PhysicalHealth	MentalHealth	I
0	No	16.60	Yes	No	No	3.0	30.0	
1	No	20.34	No	No	Yes	0.0	0.0	
2	No	26.58	Yes	No	No	20.0	30.0	
3	No	24.21	No	No	No	0.0	0.0	
4	No	23.71	No	No	No	28.0	0.0	

```
In [5]: dataset.isna().sum()
```

Out[5]:

	0
HeartDisease	0
BMI	6
Smoking	74
AlcoholDrinking	115
Stroke	21
PhysicalHealth	17
MentalHealth	14
DiffWalking	33
Sex	313
AgeCategory	44
Race	39
Diabetic	33
PhysicalActivity	16
GenHealth	493
SleepTime	219
Asthma	0
KidneyDisease	17
SkinCancer	0

dtype: int64

```
In [6]: dataset['BMI'].fillna(dataset['BMI'].mean(), inplace=True)
dataset['Smoking'].fillna(dataset['Smoking'].mode()[0], inplace=True)
dataset['AlcoholDrinking'].fillna(dataset['AlcoholDrinking'].mode()[0], inplace=True)
dataset['Stroke'].fillna(dataset['Stroke'].mode()[0], inplace=True)
dataset['PhysicalHealth'].fillna(dataset['PhysicalHealth'].mean(), inplace=True)
dataset['MentalHealth'].fillna(dataset['MentalHealth'].mean(), inplace=True)
dataset['DiffWalking'].fillna(dataset['DiffWalking'].mode()[0], inplace=True)
dataset['Sex'].fillna(dataset['Sex'].mode()[0], inplace=True)
dataset['AgeCategory'].fillna(dataset['AgeCategory'].mode()[0], inplace=True)
dataset['Race'].fillna(dataset['Race'].mode()[0], inplace=True)
dataset['Diabetic'].fillna(dataset['Diabetic'].mode()[0], inplace=True)
dataset['PhysicalActivity'].fillna(dataset['PhysicalActivity'].mode()[0], inplace=True)
dataset['GenHealth'].fillna(dataset['GenHealth'].mode()[0], inplace=True)
dataset['SleepTime'].fillna(dataset['SleepTime'].mean(), inplace=True)
dataset['KidneyDisease'].fillna(dataset['KidneyDisease'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:1: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['BMI'].fillna(dataset['BMI'].mean(), inplace=True)
```

/tmp/ipython-input-2612038968.py:2: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['Smoking'].fillna(dataset['Smoking'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:3: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['AlcoholDrinking'].fillna(dataset['AlcoholDrinking'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:4: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['Stroke'].fillna(dataset['Stroke'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:5: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['PhysicalHealth'].fillna(dataset['PhysicalHealth'].mean(), inplace=True)
```

/tmp/ipython-input-2612038968.py:6: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['MentalHealth'].fillna(dataset['MentalHealth'].mean(), inplace=True)
```

/tmp/ipython-input-2612038968.py:7: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['DiffWalking'].fillna(dataset['DiffWalking'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:8: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['Sex'].fillna(dataset['Sex'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:9: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['AgeCategory'].fillna(dataset['AgeCategory'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:10: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['Race'].fillna(dataset['Race'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:11: FutureWarning: A value is trying to be set on a

copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['Diabetic'].fillna(dataset['Diabetic'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:12: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['PhysicalActivity'].fillna(dataset['PhysicalActivity'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:13: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['GenHealth'].fillna(dataset['GenHealth'].mode()[0], inplace=True)
```

/tmp/ipython-input-2612038968.py:14: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['SleepTime'].fillna(dataset['SleepTime'].mean(), inplace=True)
```

/tmp/ipython-input-2612038968.py:15: FutureWarning: A value is trying to be set on a copy of a DataFrame or Series through chained assignment using an inplace method. The behavior will change in pandas 3.0. This inplace method will never work because the intermediate object on which we are setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try using 'df.method({col: value}, inplace=True)' or df[col] = df[col].method(value) instead, to perform the operation inplace on the original object.

```
dataset['KidneyDisease'].fillna(dataset['KidneyDisease'].mode()[0], inplace=True)
```

```
In [7]: dataset.head()
```

```
Out[7]:
```

	HeartDisease	BMI	Smoking	AlcoholDrinking	Stroke	PhysicalHealth	MentalHealth	I
0	No	16.60	Yes	No	No	3.0	30.0	
1	No	20.34	No	No	Yes	0.0	0.0	
2	No	26.58	Yes	No	No	20.0	30.0	
3	No	24.21	No	No	No	0.0	0.0	
4	No	23.71	No	No	No	28.0	0.0	

```
In [8]: dataset.isna().sum().sum()
```

```
Out[8]: np.int64(0)
```

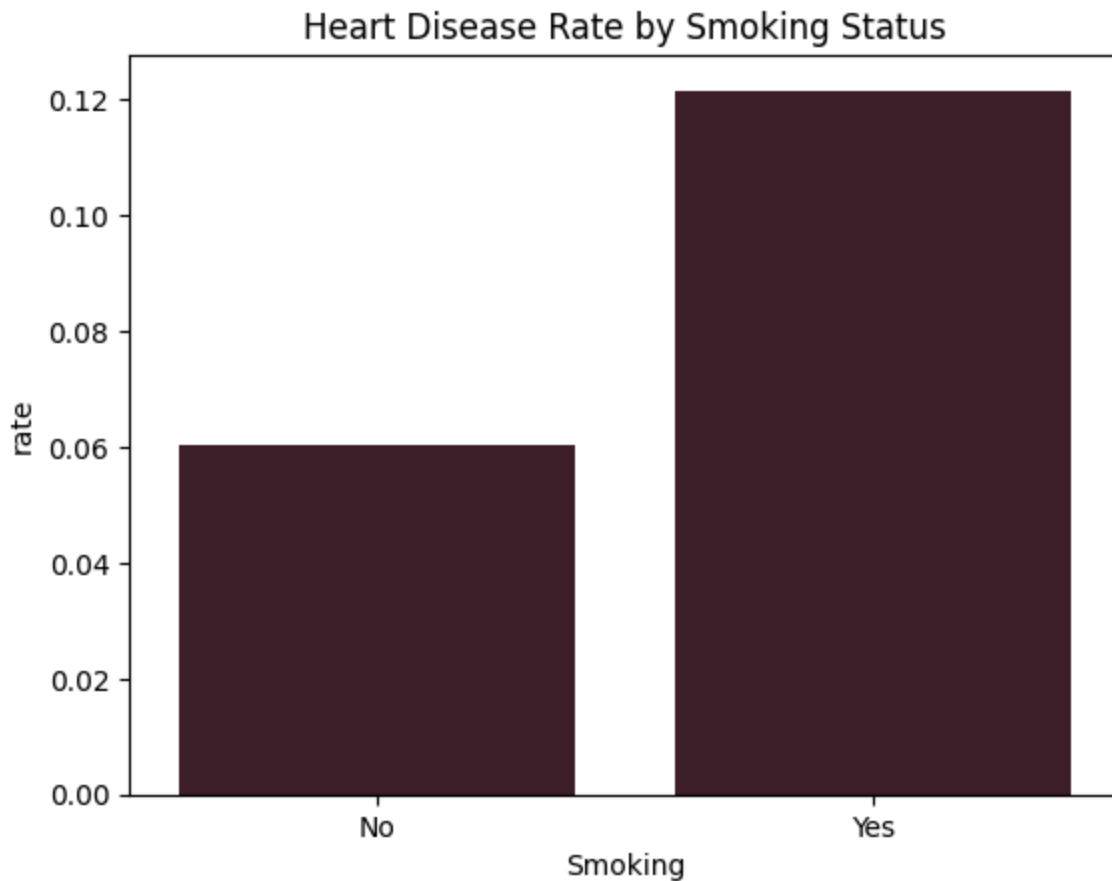
```
In [9]: dataset['HeartDisease'] = dataset['HeartDisease'].map({'Yes': 1, 'No': 0})
```

1. Heart disease against smoking habit

```
In [10]: heart_disease_vs_smoking_rate = (  
    dataset.groupby('Smoking')['HeartDisease']  
    .aggregate('mean', 'sum')  
    .reset_index(name='rate')  
)  
print(heart_disease_vs_smoking_rate)
```

	Smoking	rate
0	No	0.060346
1	Yes	0.121574

```
In [11]: sns.barplot(  
    data = heart_disease_vs_smoking_rate,  
    x = 'Smoking',  
    y = 'rate',  
    color = '#451828'  
)  
plt.title("Heart Disease Rate by Smoking Status")  
plt.show()
```



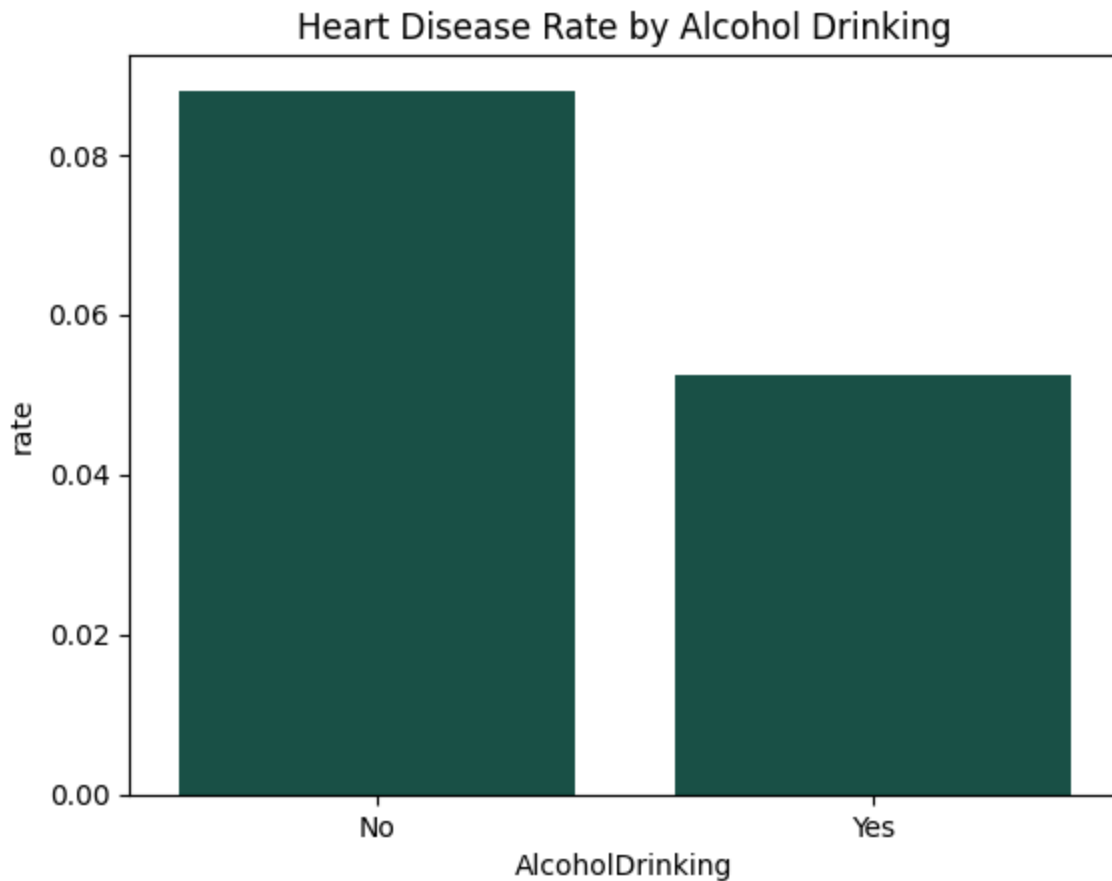
1. People who smoke have a 12.16% rate of heart disease.
2. People who don't smoke have a 6.03% rate.
3. Smoking nearly doubles the risk of heart disease in this dataset.

2. Heart Disease against alcohol consumption

```
In [12]: heart_disease_vs_alcohol_drinking = (  
    dataset.groupby('AlcoholDrinking')['HeartDisease']  
    .aggregate('mean', 'sum')  
    .reset_index(name='rate')  
)  
print(heart_disease_vs_alcohol_drinking)
```

	AlcoholDrinking	rate
0	No	0.088023
1	Yes	0.052361

```
In [13]: sns.barplot(  
    data = heart_disease_vs_alcohol_drinking,  
    x = 'AlcoholDrinking',  
    y = 'rate',  
    color = '#105e51'  
)  
plt.title("Heart Disease Rate by Alcohol Drinking")  
plt.show()
```



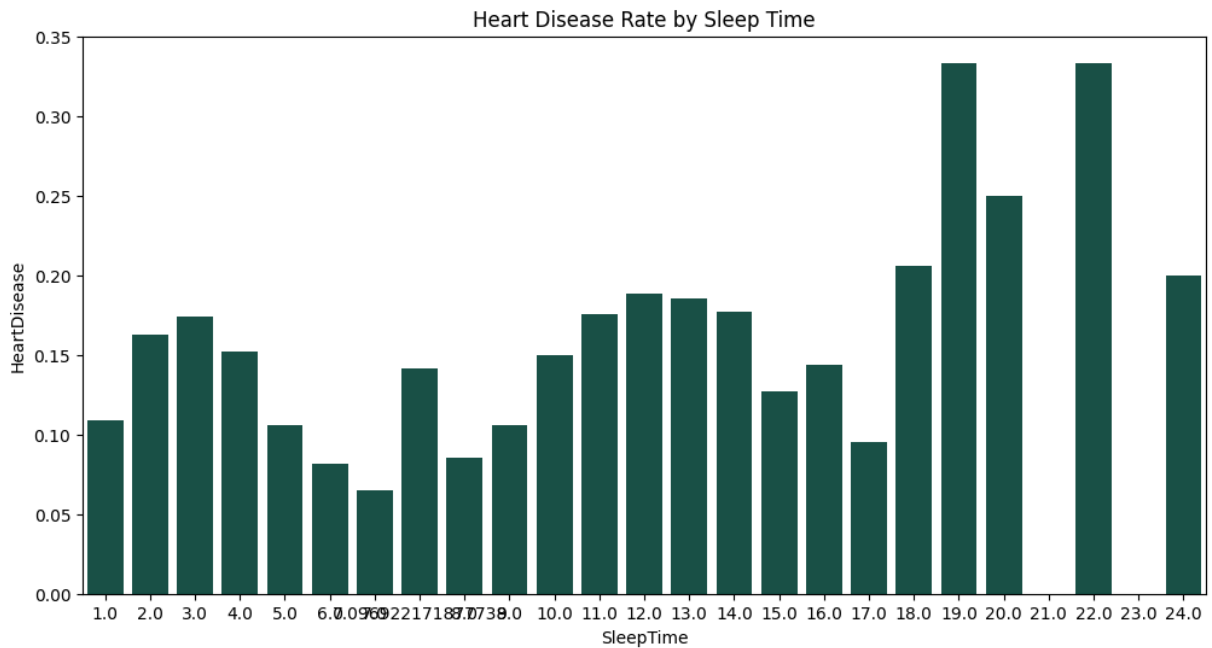
1. People who do not drink alcohol have a 8.8% rate of heart disease.
2. People who do drink alcohol have a 5.24% rate.
3. Surprisingly, the dataset suggests that alcohol drinkers have a lower heart disease rate than non-drinkers.

3. Heart Disease against sleeping pattern

```
In [14]: heart_disease_vs_sleeping_pattern = (  
    dataset.groupby('SleepTime')['HeartDisease']  
    .mean()  
    .sort_values(ascending=False)  
    .reset_index()  
)  
  
print(heart_disease_vs_sleeping_pattern)
```

	SleepTime	HeartDisease
0	22.000000	0.333333
1	19.000000	0.333333
2	20.000000	0.250000
3	18.000000	0.205882
4	24.000000	0.200000
5	12.000000	0.188748
6	13.000000	0.185567
7	14.000000	0.176955
8	11.000000	0.175904
9	3.000000	0.174284
10	2.000000	0.162643
11	4.000000	0.152168
12	10.000000	0.149422
13	16.000000	0.144068
14	7.096922	0.141553
15	15.000000	0.126984
16	1.000000	0.108893
17	5.000000	0.105798
18	9.000000	0.105608
19	17.000000	0.095238
20	8.000000	0.085616
21	6.000000	0.081302
22	7.000000	0.064643
23	21.000000	0.000000
24	23.000000	0.000000

```
In [15]: plt.figure(figsize=(12,6))
sns.barplot(
    data=heart_disease_vs_sleeping_pattern,
    x='SleepTime',
    y='HeartDisease', # the mean values column
    color='#105e51'
)
plt.title("Heart Disease Rate by Sleep Time")
plt.show()
```



1. Very low or very high SleepTime (like 1–4 hours or 18–24 hours) has higher heart disease rates.
2. Middle-range sleep (6–8 hours) shows lower heart disease rates.
3. Both insufficient and excessive sleep are linked with increased heart disease risk.
4. Most common healthy sleep range (6–8 hours) correlates with lower rates, consistent with medical studies.

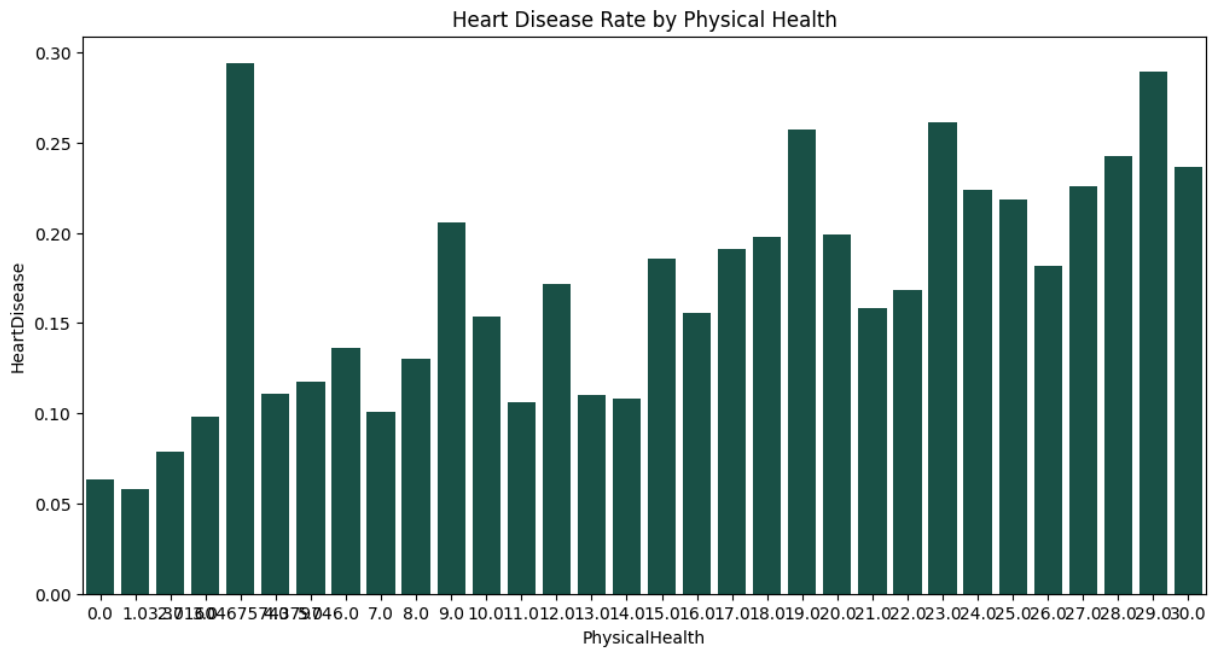
4. Heart Disease against Physical Health

```
In [16]: heart_disease_vs_physical_health = (
    dataset.groupby('PhysicalHealth')['HeartDisease']
    .mean()
    .sort_values(ascending=False)
    .reset_index()
)

print(heart_disease_vs_physical_health)
```

	PhysicalHealth	HeartDisease
0	3.371605	0.294118
1	29.000000	0.289216
2	23.000000	0.260870
3	19.000000	0.257143
4	28.000000	0.242152
5	30.000000	0.236736
6	27.000000	0.225806
7	24.000000	0.223881
8	25.000000	0.218401
9	9.000000	0.205556
10	20.000000	0.199316
11	18.000000	0.197605
12	17.000000	0.190909
13	15.000000	0.185555
14	26.000000	0.181818
15	12.000000	0.171901
16	22.000000	0.168539
17	21.000000	0.158147
18	16.000000	0.155556
19	10.000000	0.153677
20	6.000000	0.136220
21	8.000000	0.129870
22	5.000000	0.117686
23	4.000000	0.110564
24	13.000000	0.109890
25	14.000000	0.107847
26	11.000000	0.105882
27	7.000000	0.100454
28	3.000000	0.097830
29	2.000000	0.078562
30	0.000000	0.063638
31	1.000000	0.057685

```
In [17]: plt.figure(figsize=(12,6))
sns.barplot(
    data=heart_disease_vs_physical_health,
    x='PhysicalHealth',
    y='HeartDisease',
    color='#105e51'
)
plt.title("Heart Disease Rate by Physical Health")
plt.show()
```



1. PhysicalHealth is likely number of days in the past month with poor physical health.
2. Higher values > more unhealthy days > higher heart disease rates.
3. Low values (0–5 days) have lower risk, while 20+ days show much higher risk .

5. Heart Disease against various factors like Smoking, AlcoholDrinking, SleepTime, PhysicalHealth

```
In [18]: # Group by multiple features
combined_features = ['Smoking', 'AlcoholDrinking', 'SleepTime', 'PhysicalHealth']
heart_disease_combined = (
    dataset.groupby(combined_features)['HeartDisease']
    .agg(['sum', 'mean'])
    .sort_values(by='sum', ascending=False)
    .reset_index()
)

# Show top 5 combinations
top5_combined = heart_disease_combined.head(5)
print(top5_combined)
```

	Smoking	AlcoholDrinking	SleepTime	PhysicalHealth	sum	mean
0	Yes	No	8.0	0.0	2635	0.105085
1	No	No	8.0	0.0	2190	0.050503
2	Yes	No	7.0	0.0	1902	0.080916
3	No	No	7.0	0.0	1771	0.039187
4	Yes	No	6.0	0.0	1457	0.087739

```
In [19]: top5_combined['combo_readable'] = (
    "Smoking: " + top5_combined['Smoking'].astype(str) + ", " +
```

```

    "Alcohol: " + top5_combined['AlcoholDrinking'].astype(str) + ", " +
    "Sleep: " + top5_combined['SleepTime'].astype(str) + " hrs, " +
    "PhysicalHealth: " + top5_combined['PhysicalHealth'].astype(str) + " days"
)

# Sort by 'sum' descending to get bars in order
top5_combined = top5_combined.sort_values(by='sum', ascending=False)

```

/tmp/ipython-input-2903146847.py:1: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using `.loc[row_indexer,col_indexer] = value` instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

top5_combined['combo_readable'] = (

```

In [20]: top5_combined

Out[20]:

	Smoking	AlcoholDrinking	SleepTime	PhysicalHealth	sum	mean	combo_readable
0	Yes	No	8.0	0.0	2635	0.105085	Smoking: Yes, Alcohol: No, Sleep: 8.0 hrs, Phy...
1	No	No	8.0	0.0	2190	0.050503	Smoking: No, Alcohol: No, Sleep: 8.0 hrs, Phys...
2	Yes	No	7.0	0.0	1902	0.080916	Smoking: Yes, Alcohol: No, Sleep: 7.0 hrs, Phy...
3	No	No	7.0	0.0	1771	0.039187	Smoking: No, Alcohol: No, Sleep: 7.0 hrs, Phys...
4	Yes	No	6.0	0.0	1457	0.087739	Smoking: Yes, Alcohol: No, Sleep: 6.0 hrs, Phy...

In [21]:

```

# Plot horizontal bar chart
plt.figure(figsize=(12,6))
sns.barplot(
    data=top5_combined,
    x='sum',
    y='combo_readable', # readable labels
    palette='magma'
)
plt.xlabel("Number of Heart Disease Cases")
plt.ylabel("Feature Combination")

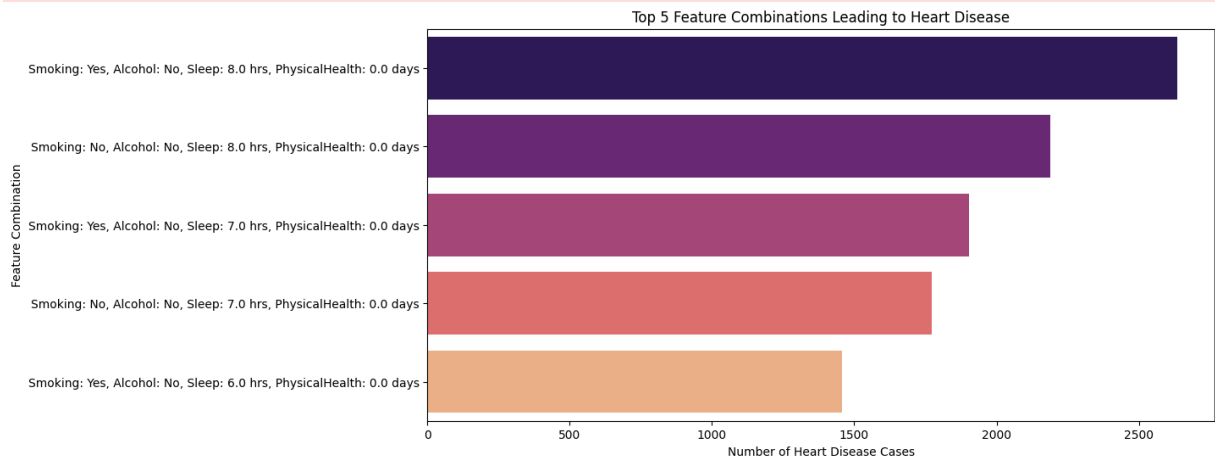
```

```
plt.title("Top 5 Feature Combinations Leading to Heart Disease")
plt.show()
```

/tmp/ipython-input-914638001.py:3: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(
```



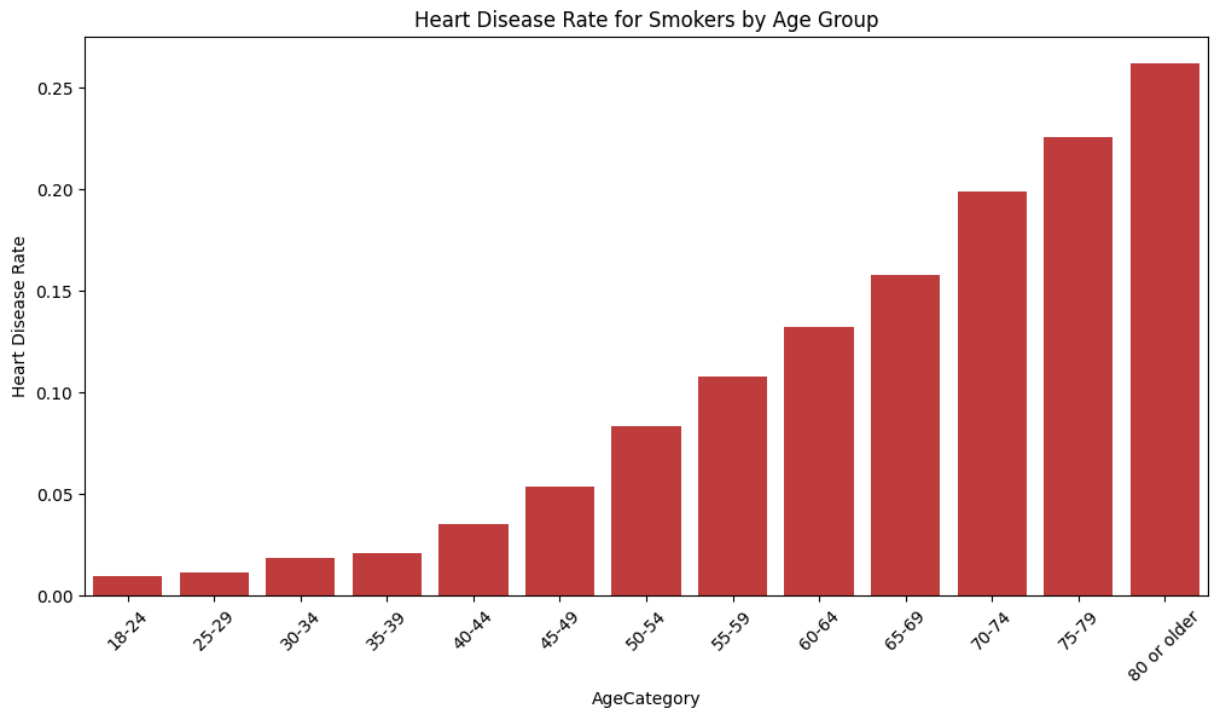
1. The highest count and risk combination is smokers who do not drink, sleep 8 hours, and have 0 poor health days: 2,633 people.
2. Second is non-smokers, non-drinkers, sleep 8 hours, 0 poor health days: 2,186 people.
3. Third is smokers, non-drinkers, sleep 7 hours, 0 poor health days, 1,901 people.
4. Smoking significantly increases risk across different sleep durations.
5. Alcohol drinking seems less prevalent in the high-risk groups here.
6. Sleep time impacts risk but not as strongly as smoking.
7. People with 0 poor physical health days still show significant risk when smoking, meaning smoking alone is a major risk factor.

6. Smoker Against Age

```
In [24]: smoke_age = (
    dataset[dataset['Smoking'] == 'Yes']
    .groupby('AgeCategory')['HeartDisease']
    .mean()
    .reset_index()
    .sort_values('AgeCategory')
)
print(smoke_age)
```

	AgeCategory	HeartDisease
0	18-24	0.009747
1	25-29	0.011454
2	30-34	0.018555
3	35-39	0.021067
4	40-44	0.035128
5	45-49	0.053307
6	50-54	0.083475
7	55-59	0.107884
8	60-64	0.132102
9	65-69	0.157847
10	70-74	0.198914
11	75-79	0.225551
12	80 or older	0.261882

```
In [25]: plt.figure(figsize=(12,6))
sns.barplot(
    data=smoke_age,
    x='AgeCategory',
    y='HeartDisease',
    color='#d62728'
)
plt.title("Heart Disease Rate for Smokers by Age Group")
plt.ylabel("Heart Disease Rate")
plt.xticks(rotation=45)
plt.show()
```



7. Drinker Against Age

```
In [26]: drink_age = (
    dataset[dataset['AlcoholDrinking'] == 'Yes']
    .groupby('AgeCategory')['HeartDisease']
    .mean()
```

```

    .reset_index()
    .sort_values('AgeCategory')
)
print(drink_age)

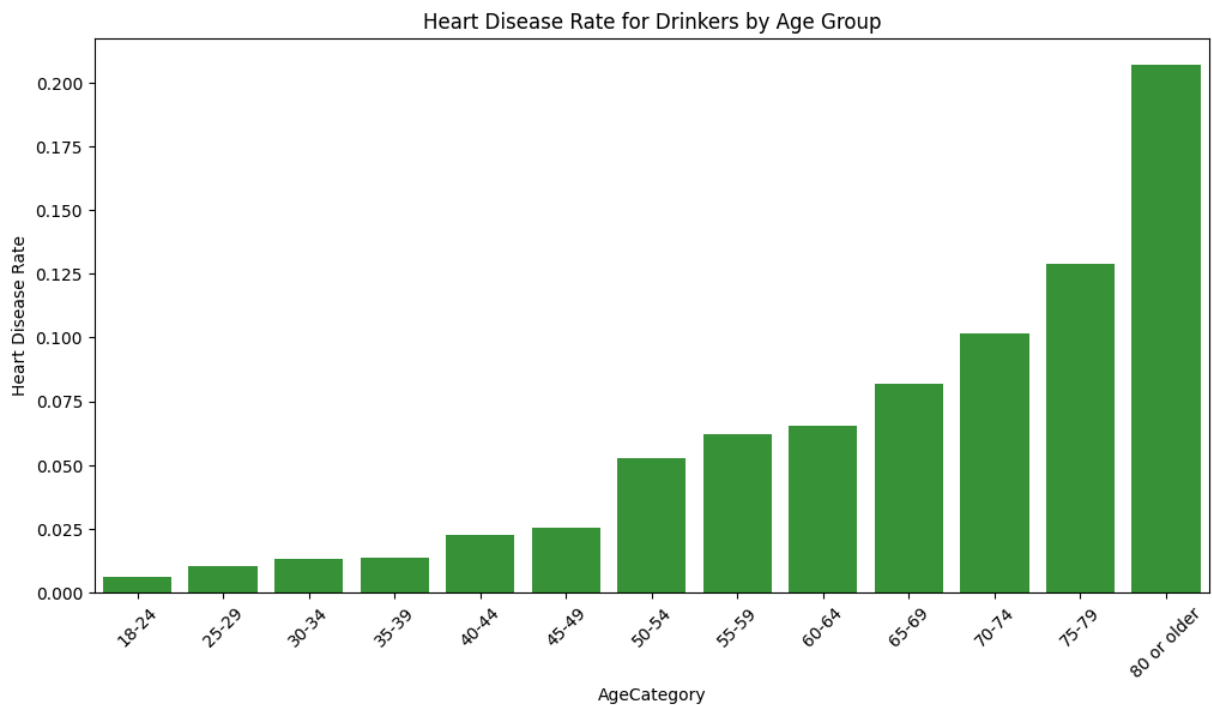
```

	AgeCategory	HeartDisease
0	18-24	0.005917
1	25-29	0.010185
2	30-34	0.013308
3	35-39	0.013699
4	40-44	0.022715
5	45-49	0.025550
6	50-54	0.052578
7	55-59	0.062218
8	60-64	0.065553
9	65-69	0.081905
10	70-74	0.101440
11	75-79	0.128889
12	80 or older	0.207048

```

In [27]: plt.figure(figsize=(12,6))
sns.barplot(
    data=drink_age,
    x='AgeCategory',
    y='HeartDisease',
    color='#2ca02c'
)
plt.title("Heart Disease Rate for Drinkers by Age Group")
plt.ylabel("Heart Disease Rate")
plt.xticks(rotation=45)
plt.show()

```



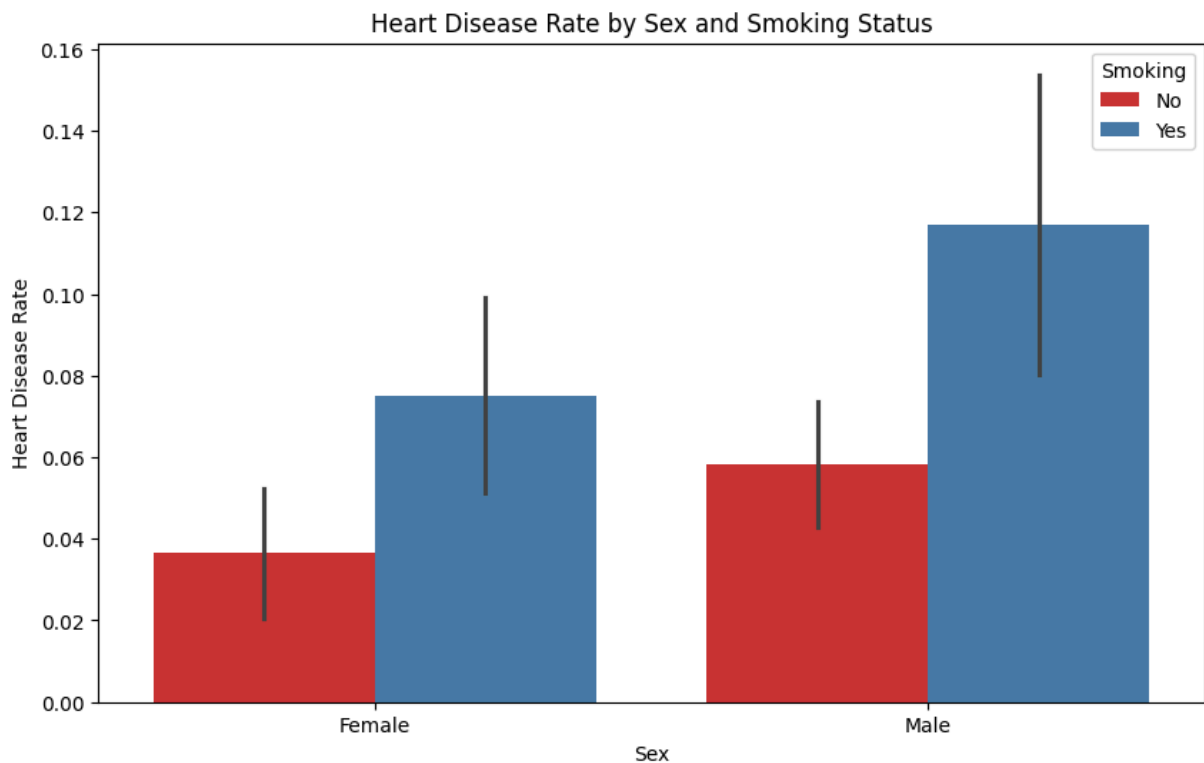
8. Smoker and Drinker Against Sex

```
In [28]: combo = (
    dataset
    .groupby(['Smoking', 'AlcoholDrinking', 'Sex'])['HeartDisease']
    .mean()
    .reset_index()
)
print(combo)
```

	Smoking	AlcoholDrinking	Sex	HeartDisease
0	No	No	Female	0.052399
1	No	No	Male	0.073667
2	No	Yes	Female	0.020619
3	No	Yes	Male	0.042723
4	Yes	No	Female	0.099058
5	Yes	No	Male	0.153744
6	Yes	Yes	Female	0.051194
7	Yes	Yes	Male	0.080240

```
In [29]: plt.figure(figsize=(10,6))
sns.barplot(
    data=combo,
    x='Sex',
    y='HeartDisease',
    hue='Smoking',
    palette='Set1'
)
plt.title("Heart Disease Rate by Sex and Smoking Status")
plt.ylabel("Heart Disease Rate")
plt.show()

plt.figure(figsize=(10,6))
sns.barplot(
    data=combo,
    x='Sex',
    y='HeartDisease',
    hue='AlcoholDrinking',
    palette='Set2'
)
plt.title("Heart Disease Rate by Sex and Alcohol Drinking Status")
plt.ylabel("Heart Disease Rate")
plt.show()
```



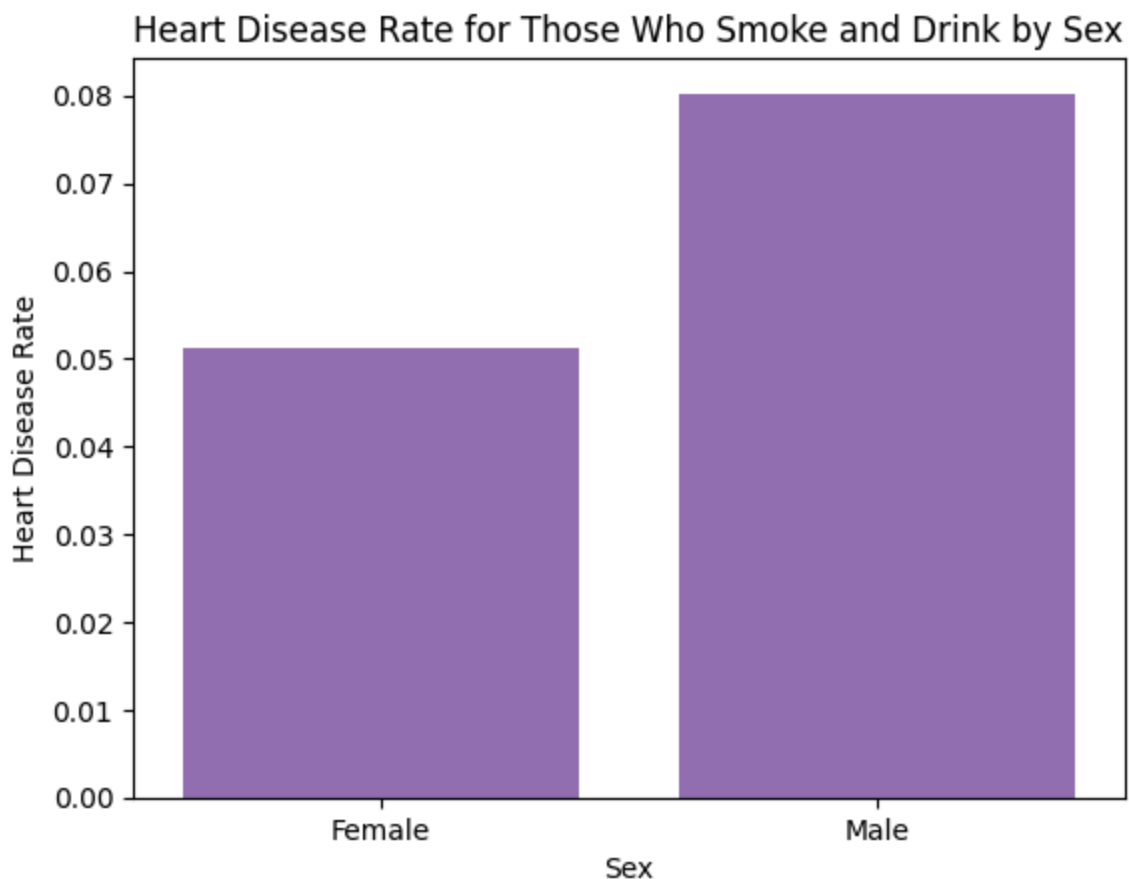
9. Combined smoker+drinker vs. sex

```
In [30]: both_smoke_drink = (
dataset[(dataset['Smoking'] == 'Yes') & (dataset['AlcoholDrinking'] == 'Yes')]
.groupby('Sex')['HeartDisease']
.mean()
.reset_index()
```

```
)  
print(both_smoke_drink)
```

```
   Sex  HeartDisease  
0  Female    0.051194  
1   Male    0.080240
```

```
In [31]: sns.barplot(  
    data=both_smoke_drink,  
    x='Sex',  
    y='HeartDisease',  
    color='#9467bd'  
)  
plt.title("Heart Disease Rate for Those Who Smoke and Drink by Sex")  
plt.ylabel("Heart Disease Rate")  
plt.show()
```



This study analyzed the impact of lifestyle and health factors on heart disease using data analysis and visualization techniques. Smoking was found to be the strongest risk factor, with smokers showing nearly double the heart disease rate compared to non-smokers. Sleep duration showed a U-shaped relationship, where both insufficient and excessive sleep increased risk, while 6-8 hours was associated with lower rates. Poor physical health days were strongly correlated with higher heart disease risk. Combined feature analysis confirmed that smoking consistently appears in the highest-risk groups. Overall, the results demonstrate how data-driven decision support systems can identify key risk patterns for cardiovascular disease.

Random Forest Algorithm

```
In [33]: from sklearn.model_selection import train_test_split
from sklearn.metrics import confusion_matrix, classification_report
from sklearn.ensemble import RandomForestClassifier
```

```
In [34]: features = [
    'BMI', 'Smoking', 'AlcoholDrinking', 'Stroke',
    'PhysicalHealth', 'MentalHealth', 'DiffWalking', 'Sex',
    'AgeCategory', 'Race', 'Diabetic', 'PhysicalActivity',
    'GenHealth', 'SleepTime', 'Asthma', 'KidneyDisease', 'SkinCancer'
]

# Encode categoricals
df_model = dataset[features + ['HeartDisease']]
df_model_encoded = pd.get_dummies(df_model, drop_first=True)
X = df_model_encoded.drop(columns=['HeartDisease'])
y = df_model_encoded['HeartDisease']
```

```
In [35]: X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)
```

```
In [48]: # use Synthetic Minority Oversampling Technique (SMOT)
from imblearn.over_sampling import SMOTE
smote = SMOTE(random_state=53)
X_train_res, y_train_res = smote.fit_resample(X_train, y_train)
clf.fit(X_train_res, y_train_res)
y_pred = clf.predict(X_test)
```

```
In [49]: cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:\n", cm)
print("\nClassification Report:\n", classification_report(y_test, y_pred))
```

Confusion Matrix:

```
[[54411  4073]
 [ 3766 1709]]
```

Classification Report:

	precision	recall	f1-score	support
0	0.94	0.93	0.93	58484
1	0.30	0.31	0.30	5475
accuracy			0.88	63959
macro avg	0.62	0.62	0.62	63959
weighted avg	0.88	0.88	0.88	63959

```
In [ ]: !jupyter nbconvert --to webpdf "D:/GithubData-Science-And-Machine-Learning-Course\D
```